DEVOIR DE CONTROLE CONTINU
JEU DE BLACKJACK

UNIVERSITÉ
CAEN
NORMANDIE

UE: Méthodes de conception
Chargé de TP : Christophe Charrier

Andrea Gjoreska, Mehmet Tuna Ozkalkanli, Mila Bucevska

Année Universitaire 2024-2025

Table des matières

1 Introduction **3**

2 Fonctionnalités principales **3**

3 Architecture MVC **3**
3.1 Le Modèle 3
3.2 Le Contrôleur 4
3.3 La Vue 4

4 Design Patterns Implémentés **4**
4.1 State Pattern 4
4.2 Strategy Pattern 4
4.3 Proxy Pattern 5
4.4 Adapter Pattern 5
4.5 Observer Pattern 5

5 Fonctionnalités Spécifiques **5**
5.1 Gestion des mains multiples (Split) 5
5.2 Système d'assurance 5
5.3 Système de statistiques 6

6 Interface Graphique **6**
6.1 Menu principal 6
6.2 Fenêtre de jeu 6

7 Difficultés Rencontrées et Solutions **7**

8 Conclusion **7**

1 Introduction

Dans le cadre de notre formation, nous avons développé un jeu de Blackjack en Java avec une interface graphique Swing. Ce projet nous a permis d'appliquer les principes de la programmation orientée objet et d'implémenter plusieurs design patterns comme MVC, Strategy, State, Proxy, Adapter et Observer pour structurer le jeu de manière efficace et modulaire.

2 Fonctionnalités principales

Notre implémentation du Blackjack comprend les fonctionnalités suivantes :

- Jeu contre un robot avec plusieurs stratégies au choix (normale, conservative, agressive, aléatoire)
- Gestion complète des actions de jeu : Hit, Stand, Double, Split, Insurance
- Système de statistiques détaillées pour suivre les performances du joueur
- Interface graphique avec affichage des cartes et animations
- Gestion de la balance du joueur et système de mises
- Support du clavier pour une meilleure expérience de jeu

3 Architecture MVC

Nous avons structuré notre application selon le patron Modèle-Vue-Contrôleur (MVC). Cette séparation des responsabilités facilite la maintenance et l'évolution du code. Le Model gère la logique du jeu, la View affiche l'interface graphique, et le Controller gère les interactions utilisateur.

3.1 Le Modèle

Le modèle contient toute la logique du jeu et gère des opérations comme le mélange du paquet, la distribution des cartes, le calcul des valeurs des mains et la détermination du gagnant. Il est organisé en plusieurs packages :

- `blackjack.model` : Contient les classes principales comme `Game`, `Player`, `PlayerHand`, `DealerHand`, `Deck`, `Card`, et les états du jeu.
- `blackjack.model.strategy` : Implémentation du pattern Strategy pour les comportements des robots.

Deck et Card : La classe `Deck` représente le paquet de cartes utilisé dans le jeu et gère le mélange et la distribution. Elle expose des méthodes comme `shuffle()` pour randomiser les cartes et `dealCard()` pour les distribuer. La classe `Card` représente une carte individuelle avec ses attributs (couleur et valeur) et fournit des méthodes pour calculer sa valeur.

Player et PlayerHand : La classe `Player` représente un joueur avec son solde, ses mises et un tableau de mains, car un joueur peut en avoir plusieurs suite à un split. La classe `PlayerHand` gère la main de cartes et fournit des méthodes pour calculer la valeur de la main, vérifier les conditions de Blackjack ou de bust, et mettre à jour la main selon les actions du joueur.

Game : La classe `Game` est au cœur du modèle et coordonne tout le déroulement du jeu. Elle gère les étapes du jeu, la gestion des joueurs, la mise à jour des états et la

vérification de l'issue de chaque tour. Cette classe interagit avec les autres composants pour mettre en œuvre les actions de mise, distribution, tours du joueur et du croupier, et paiements.

États du Jeu : Le modèle inclut un ensemble d'états qui décrivent le déroulement à chaque étape grâce à une State Machine. Ces états sont `BettingState`, `DealingState`, `PlayingState`, `DealerTurnState`, et `PayoutState` (paiement des gains). Chaque état est responsable d'une phase spécifique et garantit que le jeu suit un flux logique.

3.2 Le Contrôleur

La classe `Controller` sert de lien entre le modèle et la vue. Elle écoute les actions de l'utilisateur comme les clics sur des boutons ou les frappes au clavier et transmet ces actions au modèle. Le contrôleur met à jour le modèle et la vue en fonction des interactions, en s'assurant que l'état du jeu reste valide et cohérent. Il implémente les interfaces `ActionListener`, `MouseListener` et `KeyListener` pour gérer tous les types d'événements. Le contrôleur gère également la transition entre les différentes phases du jeu et active ou désactive certaines actions selon l'état courant.

3.3 La Vue

La vue est responsable de l'affichage graphique du jeu et de l'interface utilisateur. Elle utilise des composants Swing comme `JButton`, `JLabel` et `JPanel` pour afficher l'état du jeu, les informations des joueurs, le solde et les cartes. Les composants incluent `View` (fenêtre principale), `Menu` (démarrage et configuration), `ViewPacketVisible` (cartes visibles), `ViewPacketHidden` (paquet restant), et `Rules` (règles du jeu). La vue utilise le patron Observer pour se mettre à jour automatiquement chaque fois que l'état du modèle change, garantissant que l'interface est toujours synchronisée avec la logique du jeu.

4 Design Patterns Implémentés

Notre projet utilise plusieurs design patterns pour améliorer la qualité et la maintenabilité du code.

4.1 State Pattern

Nous avons utilisé le pattern State pour gérer les différentes phases du jeu. Chaque état encapsule les comportements spécifiques à cette phase et gère ses propres transitions possibles via la méthode `canTrans()`. Ce pattern nous permet d'éviter de multiples conditions if/else et rend le code plus maintenable. Les transitions invalides sont automatiquement prévenues.

4.2 Strategy Pattern

Nous avons implémenté quatre stratégies différentes pour le robot, permettant au joueur de choisir le niveau de difficulté :

— **BlackjackStrategy (NormalStrategy) :** Stratégie optimale basée sur les probabilités mathématiques du Blackjack. Le bot suit les règles standard et s'arrête à 17 ou plus.

— **ConservativeStrategy :** Stratégie prudente qui minimise les risques. Le bot reste souvent pour éviter de dépasser 21.
— **AggressiveStrategy :** Stratégie risquée qui double et frappe plus souvent pour maximiser les gains, même avec des valeurs de main élevées.
— **RandomStrategy :** Stratégie aléatoire où les décisions sont prises au hasard, introduisant de l'imprévisibilité.

Ce pattern permet de changer facilement le comportement du robot sans modifier le code de la classe `Player`. L'ajout d'une nouvelle stratégie se fait simplement en créant une nouvelle classe implémentant l'interface `Strategy`.

4.3 Proxy Pattern

Nous avons utilisé le pattern Proxy avec la classe `DeckProxy` pour contrôler l'accès au paquet de cartes. Le proxy nous permet de contrôler l'accès au paquet, de compter les cartes distribuées, et de recréer automatiquement un nouveau paquet lorsque celui-ci est vide. C'est une protection contre les manipulations invalides du deck.

4.4 Adapter Pattern

Nous avons utilisé le pattern Adapter avec la classe `PlayerStatisticsAdapter` pour ajouter la fonctionnalité de statistiques aux joueurs sans modifier la classe `Player`. Cet adapter permet d'ajouter le tracking des statistiques de manière transparente et de mettre à jour les données en temps réel pendant le jeu.

4.5 Observer Pattern

Le pattern Observer est utilisé pour la communication entre le modèle et la vue. Les vues implémentent `EcouteurModele` et s'enregistrent auprès du modèle. Quand le modèle change, il appelle `fireChangement()` et toutes les vues se rafraîchissent automatiquement. Ce pattern assure un couplage faible entre le modèle et la vue, facilitant les tests et la maintenance.

5 Fonctionnalités Spécifiques

5.1 Gestion des mains multiples (Split)

Lorsqu'un joueur a deux cartes de même valeur, il peut séparer sa main en deux mains distinctes. Notre implémentation gère la création d'une nouvelle main avec le même montant de mise, la distribution d'une carte supplémentaire à chaque main, le jeu séquentiel de chaque main et l'affichage graphique des deux mains côte à côte.

5.2 Système d'assurance

Quand le croupier a un As, le joueur peut prendre une assurance (maximum la moitié de sa mise). Si le croupier obtient un Blackjack, l'assurance paie 2 :1, sinon elle est perdue. Notre implémentation vérifie toutes les conditions nécessaires : le croupier doit avoir un As visible, le joueur doit avoir exactement 2 cartes, et le montant ne peut pas dépasser la moitié de la mise.

5.3 Système de statistiques

La classe `StatisticsManager` suit différentes données de performance pendant le jeu : solde initial et final, gain ou perte total, nombre de Blackjacks obtenus, victoires, défaites et nombre total de manches. Ces statistiques sont affichées à la fin du jeu dans une fenêtre récapitulative.

6 Interface Graphique

6.1 Menu principal

Le menu permet de configurer la partie avant de commencer : choix du solde initial (par défaut 1000), sélection de la stratégie du robot et accès aux règles du jeu.

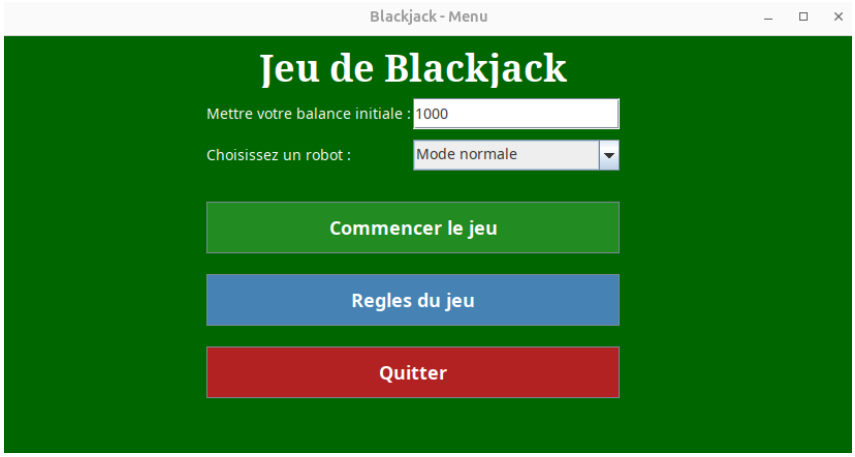

FIGURE 1 – Le menu de jeu

6.2 Fenêtre de jeu

La fenêtre principale affiche les cartes du croupier (avec une carte cachée jusqu'à la fin), les cartes du joueur humain et du robot, le paquet de cartes restantes, les boutons d'action et le champ de mise avec la balance actuelle. L'affichage des cartes est géré par les classes `ViewPacketVisible` et `ViewPacketHidden`.

Le joueur peut interagir par souris (clics sur les boutons) ou clavier (raccourcis : h=hit, s=split, t=stand, d=double, i=insurance). Des messages apparaissent pour signaler un Blackjack, proposer une assurance, afficher les erreurs et les statistiques finales.

FIGURE 2 – Fenêtre de jeu en cours

7 Difficultés Rencontrées et Solutions

Gestion du Split avec affichage graphique : Le problème était d'afficher deux mains côte à côte et de gérer le jeu séquentiel. Nous avons ajouté un attribut `indicationPlayer` dans `ViewPacketVisible` qui indique quelle main est active, et le contrôleur gère la transition.

Synchronisation Modèle-Vue : Pour assurer que la vue se met à jour correctement, nous avons utilisé rigoureusement le pattern Observer. Chaque modification du modèle appelle `fireChangement()`, ce qui déclenche automatiquement le rafraîchissement de toutes les vues enregistrées.

8 Conclusion

Ce projet de jeu de Blackjack nous a permis de mettre en pratique de nombreux concepts de la programmation orientée objet. L'utilisation des design patterns comme MVC, Strategy, State, Proxy, Adapter et Observer a grandement amélioré la structure et la maintenabilité de notre code. En respectant ces patrons, nous avons atteint une séparation claire des responsabilités, ce qui facilite les mises à jour et les extensions futures.

L'interface du jeu, implémentée avec Java Swing, est à la fois fonctionnelle et visuellement agréable. Bien que l'implémentation actuelle soit fonctionnelle, il existe des possibilités d'amélioration futures, comme l'ajout d'un mode multijoueur ou l'introduction de stratégies de bots plus avancées, qui enrichiraient encore l'expérience utilisateur.